



CẤU TRÚC DỮ LIỆU & GIẢI THUẬT

BÀI 3: Backtracking - Quay lui

PHẦN 1



Backtracking là gì?

Backtracking (quay lui) là kỹ thuật giải quyết bài toán bằng cách:

- Thử từng lựa chọn có thể (explore)
- Nếu lựa chọn đó dẫn đến giải pháp → giữ lại
- Nếu không → quay lui (backtrack), bỏ lựa chọn đó và thử lựa chọn khác

Ví dụ:

Đi qua mê cung:

- Đi thử một lối $\rightarrow$ bế tắc $\rightarrow$ quay lại ngã rẽ trước đó
- Thử lối khác $\rightarrow$ tiếp tục $\rightarrow$ cho đến khi tìm ra lối thoát

Key point:

Backtracking = Độ quy + Thử và loại bỏ

Khi nào nên dùng backtracking?

- Cần tìm tất cả các giải pháp có thể (không chỉ 1 giải pháp)
- Bài toán có nhiều lựa chọn ở mỗi bước
- Không có công thức trực tiếp hoặc thuật toán tham lam
- Bài toán tổ hợp: hoán vị, tổ hợp, tập con

Ví dụ:

- Xếp lịch (scheduling)
- Tô màu đồ thị
- Giải Sudoku

Sudoku, backtracking và tìm kiếm trong AI

Sudoku: bài toán điền số 1–9 vào bảng 9×9 , thỏa 3 ràng buộc hàng, cột, ô 3×3 .

- Mỗi state = một bảng Sudoku, mỗi bước = chọn ô trống, thử số 1–9.
- Backtracking = DFS trong không gian trạng thái, là một dạng state-space search cơ bản trong AI.



Nâng cấp backtracking Sudoku bằng heuristic

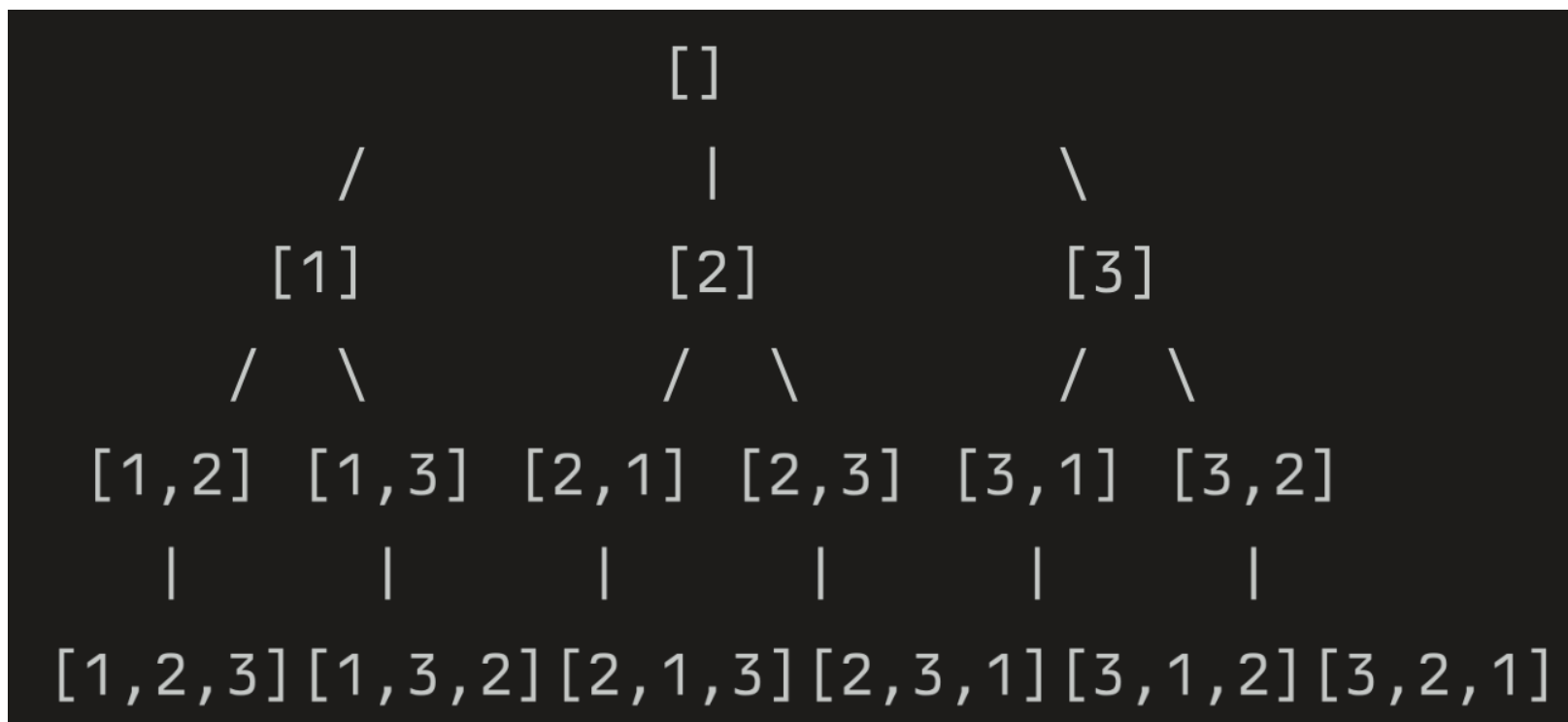
- Backtracking thuần trên Sudoku: thử số theo thứ tự, rất nhiều nhánh → chậm.
- AI thêm heuristic:
 - ✓ Chọn ô có ít lựa chọn hợp lệ nhất (MRV – most constrained variable).
 - ✓ Loại sớm các số vi phạm ràng buộc (constraint propagation).
- Kết hợp: backtracking + heuristic = search thông minh hơn, vẫn đảm bảo tìm nghiệm nhưng cắt bỏ rất nhiều nhánh.

Mô hình cây trạng thái

Giải thích:

- Mỗi nút (node) = một trạng thái của bài toán
- Mỗi cạnh (edge) = một lựa chọn (choice)
- Nút lá = trạng thái cuối cùng (có thể là giải pháp hoặc bế tắc)

Ví dụ: Tìm hoán vị của tập hợp số $[1, 2, 3]$



Key point:

Backtracking = Duyệt cây trạng thái theo DFS (Depth-First Search)

Template backtracking: Choose → Explore → Unchoose

1. Choose (Chọn):

- Thêm một lựa chọn vào trạng thái hiện tại
- Ví dụ: Thêm số 2 vào danh sách $[1] \rightarrow [1, 2]$

2. Explore (Khám phá):

- Gọi đệ quy để tiếp tục với trạng thái mới
- Đi sâu vào cây trạng thái

Template backtracking: Choose → Explore → Unchoose

3. Unchoose (Bỏ chọn):

- Xóa lựa chọn vừa thêm, quay về trạng thái ban đầu
- Ví dụ: $[1, 2] \rightarrow \text{xóa } 2 \rightarrow [1]$

Mục đích Unchoose:

Để thử các lựa chọn khác trên cùng mức độ sâu

Template Python cho backtracking

- state: Trạng thái hiện tại (ví dụ: [1, 2])
- choices: Các lựa chọn còn lại
- results: Danh sách lưu tất cả giải pháp

```
def backtrack(state, choices, results):  
    # Base case: Đã đạt giải pháp  
    if is_solution(state):  
        results.append(state.copy()) # Lưu giải pháp  
        return  
  
    # Duyệt qua tất cả lựa chọn có thể  
    for choice in choices:  
        # Kiểm tra lựa chọn có hợp lệ không  
        if is_valid(state, choice):  
            # 1. CHOOSE: Thêm lựa chọn vào trạng thái  
            state.append(choice)  
  
            # 2. EXPLORE: Đệ quy tiếp  
            backtrack(state, remaining_choices, results)  
  
            # 3. UNCHOOSE: Quay lui, bỏ lựa chọn  
            state.pop()
```

Bài toán

Cho danh sách $[1, 2, 3]$, tìm tất cả cách sắp xếp (hoán vị)

Kết quả mong đợi:

$[1, 2, 3]$, $[1, 3, 2]$, $[2, 1, 3]$, $[2, 3, 1]$, $[3, 1, 2]$, $[3, 2, 1]$

Phân tích:

- Mỗi vị trí có thể chọn từ các số còn lại
- Khi đã chọn đủ 3 số $\rightarrow$ có 1 hoán vị

Code Python - Tìm hoán vị

```
def permutations(nums):
    result = []

    def backtrack(path, remaining):
        # Base case: Đã chọn đủ số
        if len(path) == len(nums):
            result.append(path.copy())
            return

        # Duyệt qua các số còn lại
        for i in range(len(remaining)):
            # CHOOSE: Chọn remaining[i]
            path.append(remaining[i])

            # EXPLORE: Đệ quy với số còn lại (bỏ phần tử i)
            backtrack(path, remaining[:i] + remaining[i+1:])

            # UNCHOOSE: Bỏ lựa chọn
            path.pop()

    backtrack([], nums)
    return result

# Test
print(permutations([1, 2, 3]))
# Output: [[1,2,3], [1,3,2], [2,1,3], [2,3,1], [3,1,2], [3,2,1]]
```

Minh họa quá trình thực thi (Tracing)

Kết quả: $[[1,2], [2,1]]$

Bước 1: `backtrack([], [1,2])`

→ CHOOSE 1 → `path=[1], remaining=[2]`

Bước 2: `backtrack([1], [2])`

→ CHOOSE 2 → `path=[1,2], remaining=[]`

Bước 3: `backtrack([1,2], [])`

→ Base case! Lưu `[1,2]`

→ UNCHOOSE 2 → `path=[1]`

Bước 4: Quay lại bước 1

→ UNCHOOSE 1 → `path=[]`

→ CHOOSE 2 → `path=[2], remaining=[1]`

Bước 5: `backtrack([2], [1])`

→ CHOOSE 1 → `path=[2,1], remaining=[]`

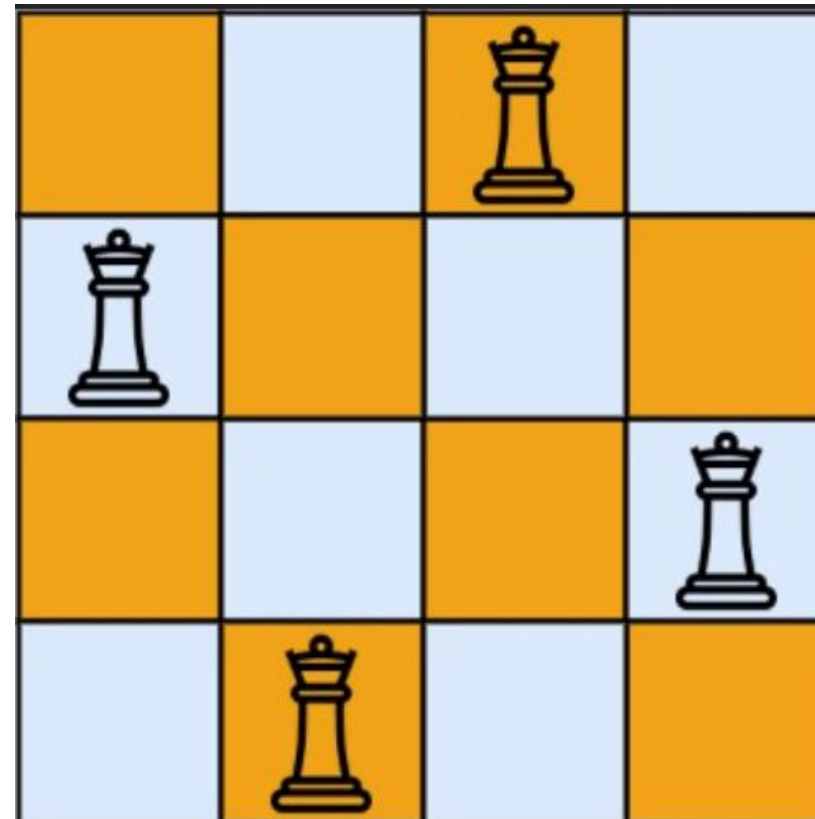
→ Base case! Lưu `[2,1]`

Bài toán N-Queens

Đặt N quân hậu trên bàn cờ $N \times N$ sao cho không có 2 quân hậu nào tấn công nhau.

Quy tắc quân hậu:

- Tấn công theo hàng ngang
- Tấn công theo cột dọc
- Tấn công theo 2 đường chéo



Thách thức:

- $N=4$: có 2 giải pháp
- $N=8$: có 92 giải pháp
- Không thể thử brute-force ($8^9 = 16777216$ khả năng!)

Phân tích cách giải N-Queens

- Mỗi hàng chỉ có đúng 1 quân hậu
- Không cần thử nhiều quân hậu trên cùng 1 hàng

Chiến lược:

- Duyệt từng hàng ($row = 0, 1, 2, \dots, N-1$)
- Ở mỗi hàng, thử đặt quân hậu vào từng cột ($col = 0$ đến $N-1$)
- Kiểm tra đặt vào cột đó có hợp lệ không (không bị tấn công)
- Nếu hợp lệ $\rightarrow$ CHOOSE $\rightarrow$ đệ quy hàng tiếp theo
- Sau đó UNCHOOSE để thử cột khác

Hàm kiểm tra vị trí hợp lệ

Giải thích:

- `board[i]`: cột của quân hậu ở hàng `i`
- Đường chéo: $|row1 - row2| == |col1 - col2|$

```
def is_safe(board, row, col, n):  
    """  
    Kiểm tra đặt quân hậu ở (row, col) có hợp lệ không  
    board: danh sách vị trí cột của quân hậu ở mỗi hàng  
    """  
  
    # Kiểm tra tất cả các hàng trước đó  
    for prev_row in range(row):  
        prev_col = board[prev_row]  
  
        # Kiểm tra cùng cột  
        if prev_col == col:  
            return False  
  
        # Kiểm tra đường chéo  
        if abs(prev_row - row) == abs(prev_col - col):  
            return False  
  
    return True
```

Code N-Queens - Hàm chính

```
def solve_n_queens(n):  
    """  
    Tìm tất cả cách đặt N quân hậu  
    """  
    result = []  
    board = [] # board[i] = cột của quân hậu ở hàng i  
  
    def backtrack(row):  
        # Base case: Đã đặt đủ N quân hậu  
        if row == n:  
            result.append(board.copy())  
            return  
  
        # Thử đặt quân hậu vào từng cột  
        for col in range(n):  
            if is_safe(board, row, col, n):  
                # CHOOSE: Đặt quân hậu ở cột này  
                board.append(col)  
  
                # EXPLORE: Đệ quy hàng tiếp theo  
                backtrack(row + 1)  
  
                # UNCHOOSE: Bỏ quân hậu  
                board.pop()  
  
    backtrack(0) # Bắt đầu từ hàng 0  
    return result
```

Hàm in bàn cờ

```
def print_board(solution, n):  
    """  
    In bàn cờ N×N với quân hậu  
    solution: danh sách [col0, col1, ..., colN-1]  
    """  
    for row in range(n):  
        line = ""  
        for col in range(n):  
            if solution[row] == col:  
                line += "Q "  
            else:  
                line += ". "  
        print(line)  
    print()  
  
# Test với 4-Queens  
solutions = solve_n_queens(4)  
print(f"Tìm thấy {len(solutions)} giải pháp:")  
  
for i, sol in enumerate(solutions):  
    print(f"Giải pháp {i+1}:")  
    print_board(sol, 4)
```

Phân tích độ phức tạp

Độ phức tạp thời gian: Trong trường hợp xấu nhất: $O(N!)$

Giải thích:

- Hàng 1: N lựa chọn
- Hàng 2: Tối đa $N-1$ lựa chọn
- Hàng 3: Tối đa $N-2$ lựa chọn
- $\rightarrow$ Tổng: $N \times (N-1) \times (N-2) \times \dots = N!$

Độ phức tạp không gian:

- Call stack: $O(N)$ (chiều sâu đệ quy = N)
- Lưu trữ board: $O(N)$

Tối ưu:

- Dùng 3 set để kiểm tra cột và chéo $\rightarrow O(1)$ thay vì $O(N)$
- Pruning (cắt tỉa) sớm các nhánh không hợp lệ

Bài toán Subset Sum (Tổng con)

- Cho mảng số nguyên nums và số nguyên target.
- Tìm tất cả tập con của nums có tổng bằng target.

Ứng dụng thực tế:

- Phân bổ ngân sách
- Đóng gói hàng hóa (bin packing)
- Chọn món ăn với tổng calo cố định

```
Input: nums = [2, 3, 5, 7], target = 7  
Output: [[2, 5], [7]]
```

```
Input: nums = [1, 2, 3, 4], target = 5  
Output: [[1, 4], [2, 3]]
```


Code Python - Subset Sum

```
def subset_sum(nums, target):
    result = []

    def backtrack(start, path, current_sum):
        # Base case: Đạt target
        if current_sum == target:
            result.append(path.copy())
            return

        # Vượt quá target → dừng (pruning)
        if current_sum > target:
            return

        # Thử thêm các số từ vị trí start
        for i in range(start, len(nums)):
            # CHOOSE
            path.append(nums[i])

            # EXPLORE (i+1 để tránh dùng lại số)
            backtrack(i + 1, path, current_sum + nums[i])

            # UNCHOOSE
            path.pop()

    backtrack(0, [], 0)
    return result

# Test
print(subset_sum([2, 3, 5, 7], 7))
# Output: [[2, 5], [7]]
```

Các điểm chính:

- Backtracking = Đệ quy + Thử và loại bỏ
- Template 3 bước: Choose $\rightarrow$ Explore $\rightarrow$ Unchoose
- Mô hình cây trạng thái giúp hình dung quá trình
- N-Queens: Ví dụ điển hình về backtracking với ràng buộc

So sánh độ phức tạp:

Bài toán	Độ phức tạp	Ghi chú
Hoán vị	$O(N! \times N)$	$N!$ hoán vị, mỗi hoán vị $O(N)$
N-Queens	$O(N!)$	Pruning giảm đáng kể
Subset Sum	$O(2^N)$	Mỗi phần tử chọn/không chọn

Backtracking trong bức tranh lớn

Kỹ thuật	Ý tưởng chính	Khi dùng	Ưu điểm	Nhược điểm
Backtracking	DFS không gian trạng thái, thử và quay lui	Bài toán tổ hợp, ràng buộc, cần liệt kê nhiều nghiệm	Dễ cài bằng đệ quy, linh hoạt, tìm được tất cả nghiệm	Thường exponential, phải thiết kế pruning tốt
Dynamic programming	Lưu kết quả bài con, tránh tính lặp lại	Có subproblem lặp, optimal substructure	Thời gian tốt hơn brute-force, có thể cho nghiệm tối ưu	Cần tìm được state/transition đúng, đôi khi tốn bộ nhớ
Heuristic search	Dùng hàm đánh giá để hướng việc tìm kiếm	Bài AI: tìm đường (A*), game, Sudoku khó, CSP phức tạp	Tìm nghiệm nhanh hơn bằng “tri thức”, có thể rất hiệu quả	Có thể không duyệt hết không gian, phụ thuộc heuristic tốt

PHẦN 2



Pruning (cắt tỉa) là gì?

Pruning = Dừng sớm các nhánh không thể dẫn đến giải pháp, thay vì thử hết tất cả.

Ví dụ: Đi mua hoa quả:

- Ngân sách: 100k
- Thấy quả đầu tiên 120k → Dừng luôn, không cần xem tiếp
- (Thay vì xem hết tất cả rồi mới quyết định)

Tại sao cần pruning?

- Backtracking có độ phức tạp exponential ($O(2^N)$, $O(N!)$)
- Với N lớn, thử tất cả là không khả thi

Ví dụ thực tế:

N	Không pruning	Có pruning
N=10	1,024 nhánh	~100 nhánh
N=20	1,048,576 nhánh	~1,000 nhánh
N=30	1 tỷ nhánh	~10,000 nhánh

Các loại điều kiện cắt tỉa

Constraint-based (Điều kiện ràng buộc):

- Kiểm tra lựa chọn có vi phạm ràng buộc không
- Ví dụ N-Queens: Quân hậu đã bị tấn công $\rightarrow$ bỏ ngay

Bound-based (Giới hạn giá trị):

- Kiểm tra đã vượt quá giới hạn chưa
- Ví dụ Subset Sum: Tổng $>$ target $\rightarrow$ dừng

Optimality-based (Tối ưu):

- Nếu giải pháp hiện tại tệ hơn best $\rightarrow$ cắt
- Ví dụ: Tìm đường đi ngắn nhất

Feasibility-based (Khả thi):

- Kiểm tra còn khả năng đạt mục tiêu không
- Ví dụ: Còn đủ số để đạt target không?

Đo lường hiệu quả của pruning

Dùng biến đếm (counter) để theo dõi:

- Total calls: Tổng số lần gọi hàm backtrack
- Pruned branches: Số nhánh đã cắt tỉa
- Solutions found: Số giải pháp tìm được

N-Queens phiên bản cơ bản (không tối ưu)

Vấn đề:

Thử tất cả N^N khả năng,

rồi mới kiểm tra hợp lệ ở cuối!

```
def solve_n_queens_no_pruning(n):  
    counter = Counter()  
    result = []  
    board = []  
  
    def backtrack(row):  
        counter.total_calls += 1  
  
        if row == n:  
            # Kiểm tra giải pháp có hợp lệ không  
            if is_valid_solution(board, n):  
                result.append(board.copy())  
                counter.solutions += 1  
            return  
  
        # Thử tất cả cột, không kiểm tra trước  
        for col in range(n):  
            board.append(col)  
            backtrack(row + 1)  
            board.pop()  
  
    backtrack(0)  
    counter.report()  
    return result
```

N-Queens với pruning (tối ưu)

Kiểm tra `is_safe()`

TRƯỚC khi đệ quy → cắt sớm!

```
def solve_n_queens_with_pruning(n):  
    counter = Counter()  
    result = []  
    board = []  
  
    def backtrack(row):  
        counter.total_calls += 1  
  
        if row == n:  
            result.append(board.copy())  
            counter.solutions += 1  
            return  
  
        for col in range(n):  
            # PRUNING: Kiểm tra trước khi thử  
            if is_safe(board, row, col, n):  
                board.append(col)  
                backtrack(row + 1)  
                board.pop()  
            else:  
                counter.pruned += 1 # Đếm nhánh đã cắt  
  
    backtrack(0)  
    counter.report()  
    return result
```

Tối ưu kiểm tra hợp lệ bằng sets

Vấn đề với cách cũ:

- `is_safe()` duyệt tất cả hàng trước $\rightarrow O(N)$
- Gọi N lần $\rightarrow$ tổng $O(N^2)$ cho mỗi cấp

Giải pháp: Dùng 3 sets

```
def solve_n_queens_optimized(n):
    result = []
    board = []
    cols = set()      # Các cột đã dùng
    diag1 = set()     # Đường chéo / (row - col)
    diag2 = set()     # Đường chéo \ (row + col)

    def backtrack(row):
        if row == n:
            result.append(board.copy())
            return

        for col in range(n):
            # Kiểm tra O(1) thay vì O(N)
            if col in cols or (row-col) in diag1 or (row+col) in diag2:
                continue # Pruning!

            # CHOOSE
            board.append(col)
            cols.add(col)
            diag1.add(row - col)
            diag2.add(row + col)

            # EXPLORE
            backtrack(row + 1)

            # UNCHOOSE
            board.pop()
            cols.remove(col)
            diag1.remove(row - col)
            diag2.remove(row + col)

    backtrack(0)
    return result
```

Tại sao row-col và row+col?

Đường chéo "/" (từ trái dưới lên phải trên):

- Các ô trên cùng đường chéo có $\text{row} - \text{col}$ bằng nhau
- Ví dụ: $(0,1), (1,2), (2,3) \rightarrow$ đều có $\text{row} - \text{col} = -1$

Đường chéo "\" (từ trái trên xuống phải dưới):

- Các ô trên cùng đường chéo có $\text{row} + \text{col}$ bằng nhau
- Ví dụ: $(0,2), (1,1), (2,0) \rightarrow$ đều có $\text{row} + \text{col} = 2$

Minh họa 4×4:

		0	1	2	3	(col)
0		+0	+1	+2	+3	(row+col)
1		+1	+2	+3	+4	
2		+2	+3	+4	+5	
3		+3	+4	+5	+6	
		0	1	2	3	
0		0	-1	-2	-3	(row-col)
1		+1	0	-1	-2	
2		+2	+1	0	-1	
3		+3	+2	+1	0	



Kết thúc